

Gradient Clipping & Accumulation, Worked Out by Hand

Capping the step length and simulating a bigger batch

Two small, surgical interventions in the training loop, computed end to end on numbers you can check in your head. *Clipping* rescales the gradient so the update step can never be longer than a chosen bound — the cure for exploding gradients in deep and recurrent nets. *Accumulation* splits one large batch into several micro-batches, sums their (rescaled) gradients, and steps once — the cure for a GPU too small to hold the batch you want. Every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes a toy model whose gradient is the two-piece vector $g_1 = [3, 4]$, $g_2 = [12]$, clips it by global norm to a budget of 1, and verifies the clipped gradient has norm exactly 1. It then takes a batch of 8 samples, splits it into 4 micro-batches of 2, and shows term by term that the accumulated micro-gradients reproduce the full-batch average gradient *exactly*.

How to read this. Clipping (Sections 1–3) and accumulation (Sections 4–6) are independent; each gets its own running example with a *Mini-example* box isolating the mechanism on the cleanest numbers. Read this right after the basic training loop topic — both tricks slot into the same `backward() → step()` sequence you already know.

Concepts covered, in order: the global gradient norm · the clip-by-norm rescale factor $\min(1, \text{max_norm}/\text{total_norm})$ · proof the clipped norm hits the budget · clip-by-value (per-component clamp) · what a micro-batch gradient is · why each micro-loss is divided by k · the term-by-term accumulation that equals the large-batch gradient · where each line sits in the loop (`zero_grad`, `clip`, `step`).

1 The setup: a gradient you can clip by hand

After `loss.backward()`, every parameter tensor `p` carries a gradient `p.grad` of the *same* shape. Clipping treats the whole collection of gradients as one long vector. To compute by hand we shrink the model to two parameter groups whose gradients are

$$g_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad g_2 = [12],$$

chosen because their norms are the clean Pythagorean values 5 and 12. Real models have millions of such components; the math is identical, just longer.

Quantity	Symbol	Value here
Gradient of group 1	g_1	$[3, 4]$
Gradient of group 2	g_2	$[12]$
Per-group norms	$\ g_1\ , \ g_2\ $	5, 12
Norm budget	<code>max_norm</code>	1
Clip type (A)	—	by global norm (rescale all)
Clip type (B)	—	by value (clamp each component)

Intuition

The gradient points in the direction of steepest *increase*; the optimizer steps a little way *against* it, and the step length is proportional to the gradient's length. A single freak batch in a deep or recurrent net can make that length explode by orders of magnitude, and one giant step wrecks the weights. Clipping bounds the *length* while keeping the *direction*: it is a speed limiter on the update, not a change of heading. That is why it tames exploding gradients in RNNs and transformers without biasing where training goes.

2 Step 1 — The global gradient norm

Clip-by-norm first measures the size of the *entire* gradient as one number: the L_2 norm of all components stacked together. Equivalently, sum the squared norms of each group and take the square root:

$$\text{total_norm} = \sqrt{\sum_p \|g_p\|^2}$$

With our two groups:

$$\begin{aligned} \|g_1\|^2 &= 3^2 + 4^2 = 25, & \|g_2\|^2 &= 12^2 = 144, \\ \text{total_norm} &= \sqrt{25 + 144} = \sqrt{169} = 13. \end{aligned}$$

This is the single scalar PyTorch's `clip_grad_norm_` computes first, and the number it returns to you (it returns the norm *before* clipping).

Mini-example — this operation in isolation

The global norm is *not* the sum of the per-group norms. Here the per-group norms are 5 and 12, summing to 17, but the global norm is $\sqrt{5^2 + 12^2} = 13$. Lengths add in quadrature, not linearly — the same Pythagorean rule that makes the hypotenuse of a 5–12 right triangle equal to 13.

3 Step 2 — The rescale factor, and proof the norm hits the budget

If `total_norm` already sits at or below the budget, clipping does nothing. Otherwise every gradient is multiplied by the *same* factor that shrinks the global norm down to exactly `max_norm`:

$$\text{scale} = \min\left(1, \frac{\text{max_norm}}{\text{total_norm}}\right)$$

Here `total_norm` = 13 > 1 = `max_norm`, so

$$\text{scale} = \frac{1}{13} = 0.077.$$

Apply it to every component:

$$g_1^* = \frac{1}{13} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.231 \\ 0.308 \end{bmatrix}, \quad g_2^* = \frac{1}{13} [12] = [0.923].$$

Verify the new global norm.

$$\|g_1^*\| = \frac{1}{13} \sqrt{25} = \frac{5}{13} = 0.385, \quad \|g_2^*\| = \frac{12}{13} = 0.923,$$

$$\text{total_norm}^* = \sqrt{0.385^2 + 0.923^2} = \frac{1}{13} \sqrt{25 + 144} = \frac{13}{13} = \boxed{1.000}.$$

Scaling every component by the constant $\frac{1}{13}$ pulls the global norm out as a factor, so the new norm is exactly $\frac{1}{13} \cdot 13 = \text{max_norm}$. The *direction* of the gradient is untouched — only its length changed.

Watch out

Order matters in the loop. Clip *after* `loss.backward()` (the gradients must exist) and *before* `optimizer.step()` (so the step uses the clipped values). `clip_grad_norm_` edits `p.grad` *in place* and returns the *pre-clip* norm — so a printed value above your budget is expected; it is telling you what it just tamed, not what it left behind.

4 Step 3 — The other knob: clip by value

Clip-by-norm rescales the whole vector by one factor. Clip-by-*value* is blunter: it clamps each component independently into $[-c, c]$, ignoring the global norm:

$$\text{clamp}(x) = \max(-c, \min(c, x)).$$

Take the flattened gradient $[3, 4, 12, -7, 1]$ with $c = 5$:

$$[3, 4, 12, -7, 1] \xrightarrow{\text{clamp to } [-5, 5]} [3, 4, 5, -5, 1].$$

Only the two out-of-range entries ($12 \rightarrow 5$, $-7 \rightarrow -5$) move; the rest pass through. The resulting vector has norm $\sqrt{3^2 + 4^2 + 5^2 + 5^2 + 1^2} = 8.718$ — value-clipping does *not* target a specific global norm, and it *changes the direction* because different components are scaled differently.

When this is the right choice

Reach for *clip-by-norm* (`clip_grad_norm_`) as the default for deep and recurrent architectures — RNNs, LSTMs, transformers — where a single direction can blow up and you want to preserve that direction while bounding the step. Reach for *clip-by-value* (`clip_grad_value_`) when individual rogue components are the problem and a hard per-element cap is acceptable. Clip-by-norm is the more common choice precisely because it keeps the descent direction intact.

5 Step 4 — Accumulation: one big batch, several micro-batches

Suppose the batch you want is $B = 8$ samples but the GPU can only hold $b = 2$ at a time. Gradient accumulation runs $k = B/b = 4$ small backward passes, lets the gradients *add up* in `p.grad`, and calls `optimizer.step()` once — producing the *same* update a single batch of 8 would have. To make it concrete, fix the per-sample gradient of each of the 8 samples (a 2-vector each):

$$G = \begin{bmatrix} 2 & -1 \\ 0 & 3 \\ 4 & 1 \\ -2 & 2 \\ 1 & 0 \\ 3 & -3 \\ -1 & 5 \\ 1 & 1 \end{bmatrix}.$$

PyTorch losses average over the batch, so the gradient the optimizer *should* see is the mean over all 8 rows:

$$g_{\text{full}} = \frac{1}{8} \sum_{i=1}^8 G_i = \frac{1}{8} [8, 8] = [1.000, 1.000]$$

Intuition

Accumulation trades wall-clock time for memory. A true batch of 8 does one forward and one backward over all 8 samples at once — fast, but it must hold 8 samples' activations in VRAM. Accumulation does 4 smaller forward/backward passes sequentially — it only ever holds 2 samples' activations, but it pays for the larger effective batch with $4\times$ the passes. Same gradient, less memory, more time.

6 Step 5 — Why each micro-loss is divided by k

Each micro-batch of $b = 2$ produces its own *mean* gradient (the loss averages over its 2 samples). With our G , the four micro-batch means are:

$$\begin{aligned} m_1 &= \frac{1}{2}([2, -1] + [0, 3]) = [1.000, 1.000], \\ m_2 &= \frac{1}{2}([4, 1] + [-2, 2]) = [1.000, 1.500], \\ m_3 &= \frac{1}{2}([1, 0] + [3, -3]) = [2.000, -1.500], \\ m_4 &= \frac{1}{2}([-1, 5] + [1, 1]) = [0.000, 3.000]. \end{aligned}$$

`backward()` adds to `p.grad`, so naively summing these gives

$$m_1 + m_2 + m_3 + m_4 = [4.000, 4.000],$$

which is $k = 4$ times too big — it is the *sum* of micro-means, but the optimizer wants their *average*. The fix is to divide every micro-loss (hence every micro-gradient) by k before the backward pass, so the accumulated sum becomes the average:

$$\sum_{m=1}^k \frac{m_m}{k} = \frac{1}{k} \sum_{m=1}^k m_m = \text{mean of the micro-means}$$

Watch out

The $\frac{1}{k}$ correction is only exact when the micro-batches are *equal in size*: then the mean of the micro-means equals the overall mean. Here every micro-batch has $b = 2$ samples, so $\frac{1}{4}(m_1 + m_2 + m_3 + m_4) = g_{\text{full}}$ exactly. If the last micro-batch is a smaller remainder, the simple $\frac{1}{k}$ scaling is slightly off and you should weight by sample counts instead.

7 Step 6 — The accumulation, term by term

Now run the loop the way the course writes it: scale each micro-loss by $\frac{1}{4}$, backward (which *adds* into `p.grad`), and watch the running sum. After step 4 we call `optimizer.step()` once and `zero_grad()`.

step m	scaled grad $m_m/4$	running sum in <code>p.grad</code>	note
1	[0.250, 0.250]	[0.250, 0.250]	accumulate
2	[0.250, 0.375]	[0.500, 0.625]	accumulate
3	[0.500, -0.375]	[1.000, 0.250]	accumulate
4	[0.000, 0.750]	[1.000, 1.000]	step + zero_grad

The final running sum is

$$\begin{bmatrix} 1.000, 1.000 \end{bmatrix} = g_{\text{full}},$$

exactly the full-batch gradient from Step 4. The accumulated micro-gradients reproduce the large-batch average gradient to the last digit — the whole point of the trick. The optimizer takes precisely the step it would have taken on a single batch of 8, but the GPU never held more than 2 samples at once.

Mini-example — this operation in isolation

Strip the model away and it is just an averaging identity. For equal groups, $\overline{\text{group means}} =$ all items:

$$\frac{m_1 + m_2 + m_3 + m_4}{4} = \frac{[4, 4]}{4} = [1, 1] = \frac{1}{8} \sum_{i=1}^8 G_i.$$

Dividing each micro-loss by k is exactly what turns the running *sum* in `p.grad` into this *mean*. Forget the $\frac{1}{k}$ and your effective learning rate is silently multiplied by k .

8 Clipping vs. accumulation, at a glance

	Gradient clipping	Gradient accumulation
Problem it solves	exploding gradients	batch too big for VRAM
Acts on	the gradient's <i>length</i>	the <i>number</i> of backward passes
Trades off	nothing (just bounds step)	compute time for memory
Where in loop	after <code>backward</code> , before <code>step</code>	<code>step/zero_grad</code> every k
Key number here	scale = $\frac{1}{13}$, norm $\rightarrow 1$	$k = 4$, grad $\rightarrow [1, 1]$
PyTorch call	<code>clip_grad_norm_(params, 1.0)</code>	<code>loss/k</code> ; step if $(i+1)\%k=0$

9 Equation cheat-sheet

Global gradient norm	$\text{total_norm} = \sqrt{\sum_p \ g_p\ ^2}$
Clip-by-norm factor	$\text{scale} = \min(1, \text{max_norm}/\text{total_norm})$
Clipped gradient	$g_p^* = \text{scale} \cdot g_p$
Resulting norm	$\min(\text{total_norm}, \text{max_norm})$
Clip-by-value	$g \mapsto \max(-c, \min(c, g))$ componentwise
Micro-batch count	$k = B/b$ (effective batch = $k \cdot b$)
Scaled micro-loss	loss $\leftarrow$ loss/ k before <code>backward</code>
Accumulated grad	$\sum_{m=1}^k m_m/k$ = full-batch mean grad
Step cadence	<code>step+zero_grad</code> when $(i+1) \bmod k = 0$

Appendix A Reference implementation

Every number above is reproducible from the following program. The first block uses PyTorch's own `clip_grad_norm`; the second is a bare accumulation loop.

```
import torch

# ----- (A) clip by global norm -----
g1 = torch.tensor([3.0, 4.0])
g2 = torch.tensor([12.0])
total = (g1.pow(2).sum() + g2.pow(2).sum()).sqrt()
print(total)                # tensor(13.)

max_norm = 1.0
scale = min(1.0, max_norm / total.item())
g1c, g2c = g1 * scale, g2 * scale
print(g1c, g2c)             # [0.2308, 0.3077] [0.9231]
new_total = (g1c.pow(2).sum() + g2c.pow(2).sum()).sqrt()
print(new_total)            # tensor(1.0000)

# the library call does exactly this, in place, returning the PRE-clip norm:
p1 = torch.zeros(2, requires_grad=True); p1.grad = g1.clone()
p2 = torch.zeros(1, requires_grad=True); p2.grad = g2.clone()
returned = torch.nn.utils.clip_grad_norm_([p1, p2], max_norm=1.0)
print(returned)             # tensor(13.) <- norm BEFORE clipping
print(p1.grad, p2.grad)     # [0.2308, 0.3077] [0.9231]

# clip by VALUE: clamp each component into [-c, c]
g = torch.tensor([3., 4., 12., -7., 1.])
print(torch.clamp(g, -5.0, 5.0)) # [3., 4., 5., -5., 1.]

# ----- (B) gradient accumulation -----
G = torch.tensor([[ 2., -1.], [ 0., 3.], [ 4., 1.], [-2., 2.],
                  [ 1., 0.], [ 3., -3.], [-1., 5.], [ 1., 1.]])

B, b = 8, 2
k = B // b                # 4 micro-batches
g_full = G.mean(0)
print(g_full)             # tensor([1., 1.]) full-batch grad

acc = torch.zeros(2)      # plays the role of p.grad
for m in range(k):
    micro_mean = G[m*b:(m+1)*b].mean(0) # loss averages over the micro-batch
    acc += micro_mean / k                # loss = loss / k -> grad / k, ADDED
    print(m + 1, (micro_mean / k).tolist(), acc.tolist())
# step 4 running sum == [1.0, 1.0] == g_full -> optimizer.step(); zero_grad()
print(torch.allclose(acc, g_full)) # True

# sketch of the real loop:
# for i, (Xb, yb) in enumerate(loader):
#     loss = criterion(model(Xb), yb) / k    # scale by 1/k
#     loss.backward()                        # gradients ACCUMULATE in p.grad
#     if (i + 1) % k == 0:
#         torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # optional
#         optimizer.step()
#         optimizer.zero_grad()              # zero ONLY every k steps
```

— end of document —